

ChadWallet

Product & Engineering One-Pager | Three-day build

LIVE

GITHUB

Collaborator invitation sent to the account for pengcheng.chen@gmail.com

1. PRODUCT AND DESIGN

ChadWallet is a responsive Solana trading product inspired by the quality bar of fomo.family. It combines a cinematic branded landing page with a complete trading workspace: live token discovery, universal search, candlestick charts, holders, live trades, wallet controls, deposits, positions, and Jupiter-powered swap routing.

Design process: The landing page was rebuilt twice during the sprint. The second rebuild replaced the initial direction with a higher-fidelity ChadWallet visual system, better motion and depth, optimized assets, real token rails, and improved mobile and desktop behavior.

Stack: Next.js, React, Tailwind CSS, Privy, Solana, Alchemy RPC, Jupiter, BirdEye, GeckoTerminal, Supabase, Cloudflare Workers, TradingView Lightweight Charts, and Vercel.

2. AI TOOLS AND TIMELINE

- **Codex:** codebase analysis, architecture, implementation, API integration, debugging, performance optimization, testing, Git operations, and deployment.
- **ChatGPT + Gemini:** asset generation, visual ideation, background removal, and image manipulation.

Task	Time
Next.js foundation and deployment setup	3h
Landing page v1, branding, responsive layout, and token rails	5h
Trading workspace, charts, market activity, and responsive panels	6h
Authentication, Solana wallet, account, deposit, and swap flows	4h
Real market data, universal search, and chart reliability	4h
Landing page v2 high-fidelity rebuild and asset refinement	4h
Performance optimization, QA, Cloudflare, Vercel, and documentation	3h

Total: approximately 29 focused hours across three days

3. ISSUES AND HOW THEY WERE FIXED

- **Market data reliability:** BirdEye could be unavailable, throttled, or sparse for new tokens. I added Jupiter, GeckoTerminal, and DexScreener fallbacks with caching, retries, and honest unavailable states instead of synthetic data.
- **Selective token search:** The initial search covered a curated list. I rebuilt it around Jupiter's live token universe with mint-address lookup, first-character queries, debouncing, and same-origin API routes.
- **Missing charts for arbitrary tokens:** I added canonical pool discovery, base/quote orientation support, cached pool resolution, and retry handling for provider rate limits.
- **Slow initial loading:** I moved the landing shell server-side, lazy-loaded authentication, optimized media delivery, and reduced unnecessary client hydration.

4. HOW THE PROJECT CAN BE IMPROVED

- Complete repeated funded-mainnet swap tests and automate transaction receipt verification.
- Add WebSocket streaming, indexed holder analytics, alerting, and provider-health monitoring.
- Integrate the licensed TradingView Charting Library when access is granted.
- Expand end-to-end tests for authentication, deposits, swaps, mobile layouts, and provider failures.
- Add production security hardening, observability, key rotation, and Apple sign-in credentials.